SOFTWARE REQUIREMENTS SPECIFICATION

for

ClassLink

A BLE Based Classroom Attendance System

Version 1.0

Prepared by: Vemula Manvi Smaran (1024160111)
Vrishank Nehru (1024160109)
Mahi Singh (1024160102)
Khagendra Saini (1024160123)

Submitted to: Dr. Raghav B. Venkataramaiyer

Course: UCS503P Software Engineering Project
Institute: Thapar Institute of Engineering and Technology

Date: October 2026

Document Information

Version	Status	Notes
1.0	Pilot release specification	First complete SRS. Backend implemented and tested. Mobile integration in progress.

Requirement status legend. *Done* means implemented and covered by an automated test in this repository. *POC* means present in the offline proof of concept only. *Planned* means specified but not yet implemented. The status column in Section 3.2 is the authoritative record of what exists today.

Contents

1 Introduction **3**

1.1 Purpose . . . 3

1.2 Intended Audience and Reading Suggestions . . . 3

1.3 Project Scope . . . 3

2 Overall Description **4**

2.1 Product Perspective . . . 4

2.2 User Classes and Characteristics . . . 4

2.3 Product Functions . . . 5

2.4 Operating Environment . . . 5

2.5 Design . . . 5

2.6 Assumptions and Dependencies . . . 5

3 System Features **6**

3.1 Description and Priority . . . 6

3.2 Functional Requirements . . . 6

3.2.1 F1. Accounts and roles . . . 6

3.2.2 F2. Classrooms and enrollment . . . 6

3.2.3 F3. Device binding . . . 7

3.2.4 F4. Sessions and rotating tokens . . . 7

3.2.5 F5. Attendance claims . . . 7

3.2.6 F6. Live roster and corrections . . . 8

3.2.7 F7. Export . . . 8

3.2.8 F8. Audit . . . 9

3.2.9 F9. Offline proof of concept mode . . . 9

4 Other Nonfunctional Requirements **9**

4.1 Performance Requirements . . . 9

4.2 Security Requirements . . . 9

4.3 Software Quality Attributes . . . 10

4.4 Business Rules 10

5 Other Requirements **11**

5.1 Deployment 11

5.2 Legal and Privacy 11

5.3 Documentation 11

A Glossary **12**

B API Summary **12**

C Rejection Reasons **12**

D Evaluation Plan and Data Schema **13**

E Test Traceability **13**

1 Introduction

1.1 Purpose

This document specifies the requirements for ClassLink version 1.0, a classroom attendance system that replaces manual roll call with a Bluetooth Low Energy (BLE) proximity check and a server authoritative record. It defines what the system does, the constraints it works within, and how the team will judge that it works.

The specification covers the whole system: the Android application used by teachers and students, the backend service, and the database. It also records which requirements the repository satisfies today, so a reader never has to guess whether a feature exists.

1.2 Intended Audience and Reading Suggestions

Reader	Start with	Then read
Course instructor and evaluators	1.3 Project Scope, 3.1 Description and Priority	4.2 Security Requirements, Appendix D (evaluation plan)
Developers on the team	2.1 Product Perspective, 3.2 Functional Requirements	Appendix B (API summary), Appendix E (test traceability)
Testers	3.2 Functional Requirements, Appendix C (rejection reasons)	Appendix E, Appendix D
Pilot teachers and administrators	2.2 User Classes, 5.1 Deployment	4.4 Business Rules

1.3 Project Scope

Manual attendance is slow, easy to proxy, and hard to audit. ClassLink lets a teacher's phone broadcast a short lived BLE token during a session. Enrolled students' phones detect the token, collect several signal strength samples, and send a signed attendance claim to a server. The server decides whether the claim counts.

In scope for version 1.0

- Teacher and student roles on Android, with sign in through Supabase Auth.
- Classrooms, join codes, and enrollment.
- One bound signing key per student device.
- Attendance sessions with rotating BLE tokens and a per session proximity policy.
- Server side validation of every claim, with a machine readable reason for every rejection.
- A live teacher roster, manual corrections with a mandatory reason, and CSV export.
- An audit trail for security relevant actions.
- Evaluation on real devices in a real classroom.

Out of scope for version 1.0

- Proof that the human owner of a phone is present. ClassLink verifies that a bound, locally authenticated device observed a valid classroom beacon under the configured policy. It does not claim more.
- Bluetooth mesh or any student to student relay.
- Tested iOS support. Some iOS code paths exist in the proof of concept but no iOS device test evidence exists.
- Integration with learning management systems, grading, or timetables.
- Cross course attendance percentages and analytics dashboards.

Goal. Cut the time spent on attendance to seconds per student while producing an auditable record. The primary success measure is detection success rate on labelled positions in a real room (Appendix D).

2 Overall Description

2.1 Product Perspective

ClassLink grew from an offline proof of concept (the `code/` Flutter app) that proved a teacher phone can advertise a session over BLE and a student phone can check in by proximity. Version 1.0 adds a server so the result no longer depends on trusting the student's phone.

Component	Technology	Responsibility
Mobile app (teacher role)	Flutter, Android first	Create classrooms, start and end sessions, advertise the rotating token, show the live roster, correct and export
Mobile app (student role)	Flutter, Android first	Join classrooms, bind a device key, scan for tokens, sample RSSI, sign and submit claims
Backend API	FastAPI (Python 3.11+)	Authenticate users, own the session and token lifecycle, validate claims, serve roster and export
Database	Supabase Postgres	Store classrooms, enrollments, sessions, challenges, attendance, audit events. Row level security protects direct access
Identity	Supabase Auth	Issue user JWTs
BLE channel	Teacher advertises, student scans	Carry a 17 byte token with no personal identifier

Data flow for one check in:

1. The teacher app asks the API for the current token (`GET /v1/sessions/{id}/token`) every 15 seconds and advertises it.
2. The student app scans, reads the token, and requests a challenge (`POST /v1/attendance/challenge`). The server checks the token, enrollment, and device binding, then returns a single use nonce and the session policy.
3. The student app samples RSSI for the session window, takes the median, signs a canonical claim string with the device key, and submits it (`POST /v1/attendance/claim`).
4. The server verifies the signature and policy, records attendance atomically, and returns the result. The teacher roster shows the new record.

2.2 User Classes and Characteristics

User class	Description	Technical skill	Notes
Teacher	Runs sessions for classrooms they own. Receives the role only if their email is on the server allowlist	Low	Holds the phone that advertises. Needs working BLE peripheral mode
Student	Joins classrooms with a join code and checks in during sessions	Low	Binds one device. Uses the phone they bring to class
Administrator	Deploys the backend, manages secrets, edits the teacher allowlist	High	Not an in app role. Works through configuration and the Supabase dashboard
Evaluator	Reads exported CSV files and evaluation data	Medium	No application access needed

2.3 Product Functions

1. **Accounts and roles.** Sign in, create a profile, receive a role.
2. **Classrooms and enrollment.** Create a classroom, share a join code, enroll.
3. **Device binding.** Register the key that signs a student's claims.
4. **Sessions and rotating tokens.** Start and end a session, issue short lived BLE tokens.
5. **Attendance claims.** Challenge, sign, validate, record.
6. **Live roster and corrections.** Watch check ins, fix mistakes with a recorded reason.
7. **Export.** Download a session as CSV.
8. **Audit.** Record every security relevant action.
9. **Offline proof of concept mode.** Keep the original no server demo available.

2.4 Operating Environment

- **Mobile.** Android 8.0 (API 26) or later with a Bluetooth LE radio. The teacher device must support BLE peripheral advertising, which depends on hardware and vendor firmware. This is why testing happens on physical devices only.
- **Backend.** Linux container running Python 3.11 or later behind HTTPS.
- **Database.** Supabase Postgres (PostgreSQL 15 or later) with the `auth` schema provided by Supabase.
- **Network.** Phones need internet access during check in. The BLE exchange itself needs no network.
- **Development.** Flutter stable (Dart 3.9 or later), Ruff, MyPy, Pytest, GitHub Actions.

2.5 Design

These constraints bind the design and implementation.

ID	Constraint
CON-01	Android is the primary tested platform. Abstractions stay portable but the project makes no claim about untested platforms.
CON-02	The BLE payload carries at most 20 bytes of application data and no student or teacher identifier. The token is 17 bytes.
CON-03	The teacher advertises and the student scans. There is no mesh and no student to student relay.
CON-04	The server is authoritative. A client's own decision never creates an attendance record.
CON-05	The token HMAC secret, database credentials, and Supabase service keys exist only on the server. Only `.env.example` files enter the repository.
CON-06	The stack is Flutter, FastAPI, and Supabase Postgres.
CON-07	The system treats RSSI as noisy evidence, never as exact distance. It decides on the median of several samples, and the threshold and sample policy belong to each session.
CON-08	Clients write data only through the API. Row level security gives clients read only access to their own rows.
CON-09	Evaluation figures come only from recorded real runs. No figure appears in a report before the raw data exists in the repository.

2.6 Assumptions and Dependencies

- Students attend with the phone they bound. A student who changes phones rebinds, and the change is audited.
- Supabase issues HS256 signed JWTs verified with the project JWT secret. Projects that use asymmetric signing keys need JWKS verification, which version 1.0 does not implement.
- Server and phone clocks stay within about a minute of each other. The server uses its own clock for every freshness decision.

3 System Features

3.1 Description and Priority

Rank	Feature	Priority	Why it ranks here
1	Attendance claims (F5)	High	The core value of the product
2	Sessions and rotating tokens (F4)	High	Claims have nothing to validate without it
3	Device binding (F3)	High	Defends against forged claims and tampered readings
4	Accounts and roles (F1)	High	Every other feature depends on identity
5	Classrooms and enrollment (F2)	High	Defines who may attend which session
6	Live roster and corrections (F6)	High	The teacher needs to see and fix the record
7	Audit (F8)	Medium	Makes the record defensible
8	Export (F7)	Medium	Gets data out for reporting
9	Offline proof of concept mode (F9)	Low	Keeps the demo path alive

3.2 Functional Requirements

Priorities use MoSCoW: **M**ust, **S**hould, **C**ould.

3.2.1 F1. Accounts and roles

ID	Requirement	Pri	Status
FR-ACC-01	The API shall accept only requests carrying a valid, unexpired Supabase JWT with the expected audience, and answer every other request with HTTP 401.	M	Done
FR-ACC-02	Each user shall have one profile with the role teacher or student. The teacher role shall come only from a server side email allowlist. Every other account shall become a student.	M	Done
FR-ACC-03	A student profile shall require a roll number.	M	Done
FR-ACC-04	Students shall not reach teacher operations and teachers shall not reach student operations. The API shall answer with HTTP 403.	M	Done
FR-ACC-05	Creating a profile twice shall return the existing profile unchanged.	S	Done

3.2.2 F2. Classrooms and enrollment

ID	Requirement	Pri	Status
FR-CLS-01	A teacher shall create a classroom. The system shall issue a unique six character join code that avoids look alike characters.	M	Done
FR-CLS-02	A student shall join a classroom with its join code. Joining again shall do no harm. An unknown code shall return HTTP 404.	M	Done
FR-CLS-03	A teacher shall list the classrooms they own and a student shall list the classrooms they joined.	M	Done

3.2.3 F3. Device binding

ID	Requirement	Pri	Status
FR-DEV-01	A student shall bind one device signing key, an ECDSA P-256 public key encoded as base64 SubjectPublicKeyInfo DER. The system shall reject keys of any other type or encoding.	M	Done
FR-DEV-02	Binding a new key shall revoke the previous key in the same transaction and shall write an audit event.	M	Done
FR-DEV-03	The private key shall never leave the device, and the app shall sign a claim only after local biometric or device credential authentication.	M	Planned

3.2.4 F4. Sessions and rotating tokens

ID	Requirement	Pri	Status
FR-SES-01	A teacher shall start a session for a classroom they own, with an RSSI threshold (default -75 dBm, range -100 to -30), a sample window (default 4 seconds, range 1 to 30), and a minimum sample count (default 3, range 1 to 50).	M	Done
FR-SES-02	A classroom shall have at most one active session. A second start shall return HTTP 409.	M	Done
FR-SES-03	A teacher shall end a session. After that the system shall reject its tokens, challenges, and claims.	M	Done
FR-SES-04	The system shall issue a rotating token of 17 bytes: version (1), session alias (4), epoch (4), and an 8 byte HMAC SHA 256 tag. The default epoch is 15 seconds and the configurable range is 5 to 120 seconds.	M	Done
FR-SES-05	The token shall carry no student or teacher identifier.	M	Done
FR-SES-06	The teacher app shall fetch a fresh token each epoch and advertise it.	M	Planned
FR-SES-07	A session shall belong to its classroom's teacher alone. Any other teacher shall receive HTTP 404 so session identifiers do not leak.	M	Done

3.2.5 F5. Attendance claims

ID	Requirement	Pri	Status
FR-ATT-01	The student app shall scan for the ClassLink service, read the token, and collect RSSI samples for the session's sample window.	M	Planned
FR-ATT-02	On a challenge request the server shall verify the token tag, token freshness (current epoch, up to one epoch old, or one ahead for clock skew), that the session is active, that the student is enrolled, and that the student has a bound device. It shall then issue a single use nonce valid for 120 seconds together with the session policy.	M	Done
FR-ATT-03	A claim shall contain the nonce, token, median RSSI, sample count, observation time, and an ECDSA signature over the canonical string `classlink-claim-v1` joined with session id, nonce, token, median RSSI, sample count, and observation time, separated by newlines.	M	Done

ID	Requirement	Pri	Status
FR-ATT-04	The server shall accept a claim only if the nonce belongs to the caller and is unused and unexpired, the token matches the challenge, the signature verifies against the active device key, the observation time lies within 60 seconds of server time, the sample count meets the session minimum, and the median RSSI meets the session threshold.	M	Done
FR-ATT-05	A student shall have at most one attendance record per session. A claim made with a new nonce after a record exists shall be rejected as a duplicate.	M	Done
FR-ATT-06	Repeating an accepted claim shall return the stored record and shall create no second record.	M	Done
FR-ATT-07	A rejection caused by policy (too few samples, weak signal) shall leave the nonce usable so the student can move closer and retry until it expires. Acceptance shall consume the nonce atomically.	S	Done
FR-ATT-08	Every rejection shall carry a reason from the set in Appendix C, returned with HTTP 409.	M	Done
FR-ATT-09	The student app shall show the server's result and explain each rejection reason in plain language.	M	Planned
FR-ATT-10	The student app shall stop scanning and report failure after 90 seconds without a valid session.	S	POC (no automated test yet)

3.2.6 F6. Live roster and corrections

ID	Requirement	Pri	Status
FR-ROS-01	A teacher shall view every enrolled student of a session with status (present, absent, or not yet recorded), source, median RSSI, and time.	M	Done
FR-ROS-02	The teacher app shall refresh the roster at least every 5 seconds while a session runs.	S	Planned
FR-COR-01	A teacher shall set an enrolled student present or absent. A reason of at least five characters shall be mandatory. The correction shall replace any BLE record and carry the source manual correction.	M	Done
FR-COR-02	Each correction shall write an audit event holding the actor, the student, the previous and new status, and the reason.	M	Done

3.2.7 F7. Export

ID	Requirement	Pri	Status
FR-EXP-01	A teacher shall download a session as CSV with the columns Classroom, Session Start, Name, Roll Number, Status, Source, Median RSSI (dBm), and Recorded At. Every enrolled student shall appear. A student without a record shall show status absent and source no_record.	M	Done
FR-EXP-02	The export shall neutralise cells that begin with `=`, `+`, `-`, `@`, tab, or carriage return so a student's name cannot run a spreadsheet formula.	M	Done
FR-EXP-03	Each export shall write an audit event.	S	Done

3.2.8 F8. Audit

ID	Requirement	Pri	Status
FR-AUD-01	The system shall record audit events for classroom creation, device binding, session start, session end, accepted claims, rejected claims (with reason and pipeline stage), manual corrections, and exports.	M	Done
FR-AUD-02	Clients shall have no read or write access to the audit log.	M	Done

3.2.9 F9. Offline proof of concept mode

ID	Requirement	Pri	Status
FR-POC-01	The original offline mode shall remain: the teacher advertises a six digit code, the student checks in by a GATT write, and the teacher exports CSV on the device. The app shall label this mode as not authoritative.	C	POC

4 Other Nonfunctional Requirements

4.1 Performance Requirements

These are targets. The pilot measures them, and the project reports only measured values.

ID	Requirement	Target
NFR-PERF-01	Discovery latency: teacher starts advertising to first valid student packet, same room	5 seconds or less
NFR-PERF-02	Attendance completion latency: student scan start to server result	10 seconds or less
NFR-PERF-03	API latency for the challenge and claim endpoints at 50 concurrent students	p95 of 500 ms or less
NFR-PERF-04	Capacity: one classroom with 300 enrolled students and a session of up to 150 concurrent check ins	No errors from capacity limits
NFR-PERF-05	Roster read for a session of 300 students	p95 of 500 ms or less

4.2 Security Requirements

ID	Requirement
NFR-SEC-01	All traffic between apps and the API shall use HTTPS in production.
NFR-SEC-02	The token HMAC secret and database service credentials shall exist only on the server. CI shall fail a build that tracks a `.env` file or a keystore.
NFR-SEC-03	Row level security shall be enabled on every table. Clients shall read only their own data and write nothing. Challenges and audit events shall be unreadable to clients.
NFR-SEC-04	The system shall compare token tags in constant time.
NFR-SEC-05	A signed claim shall bind the session, nonce, token, median RSSI, sample count, and observation time, so editing any value after signing invalidates the signature.
NFR-SEC-06	The system shall verify JWT signature, expiry, and audience on every request.

ID	Requirement
NFR-SEC-07	The system shall store only name, roll number, email, a device public key, and attendance data. It shall not collect location. The Android manifest shall declare `neverForLocation` on the Bluetooth scan permission.

Threat model. ClassLink resists casual proxy attendance. It does not resist every attack.

Threat	Mitigation	Residual risk
Forged claim	Signature with a bound device key	None for unbound keys
Edited RSSI or sample count	Fields sit inside the signature	None after signing
Replayed claim	Single use nonce, idempotent result	None
Forged or guessed token	HMAC tag, 8 bytes	2^-64 per guess
Photographed or recorded token	15 second epochs, one epoch of grace	A relayed live token still works for a few seconds
Friend checks in with the student's phone	Local biometric gate (planned), bound device	A cooperating friend who can unlock the phone
Relay attack (token forwarded to a remote phone)	Short epochs, RSSI policy, audit trail	Not eliminated
Student claims to be teacher	Allowlist grants the role	Anyone who controls an allowlisted email
Stolen JWT	Supabase token expiry	Window until expiry
Compromised device key	Hardware backed, non exportable key (planned)	Rooted devices

4.3 Software Quality Attributes

Attribute	Requirement
Reliability	Attendance writes shall be idempotent under retry and atomic. Two racing claims for one student shall produce one record.
Availability	The API shall expose `/healthz` (process) and `/readyz` (database). The pilot targets best effort availability during class hours.
Maintainability	The backend shall pass Ruff and strict MyPy in CI. Every functional requirement marked Done shall have an automated test (Appendix E).
Testability	Time shall be injectable so tests can exercise token expiry without waiting. Tests shall run against a real Postgres.
Portability	The backend shall run from one container image configured by environment variables.
Usability	A student shall complete a check in with one tap after joining a classroom. Every rejection shall have a plain language explanation.
Observability	Every accepted and rejected claim shall leave an audit event that evaluators can query.

4.4 Business Rules

ID	Rule
BR-01	A student has at most one attendance record per session.
BR-02	Only the classroom's teacher can see or change that classroom's sessions and attendance.
BR-03	A manual correction always carries a reason and an actor.
BR-04	An ended session never restarts. A new session gets a new alias and new tokens.

ID	Rule
BR-05	A student has one active device. Replacing it revokes the old key and leaves an audit event.
BR-06	An administrator alone decides who becomes a teacher, through the allowlist.
BR-07	Evaluation reports use only values computed from recorded raw data.

5 Other Requirements

5.1 Deployment

- **OTHER-DEP-01.** The backend shall build into a container image with a non root user and run under Uvicorn.
- **OTHER-DEP-02.** Configuration shall come from `CLASSLINK_*` environment variables. The `.env.example` file in the backend folder documents every variable.
- **OTHER-DEP-03.** The administrator shall apply `backend/migrations/0001_init.sql` to the Supabase project before the first start.
- **OTHER-DEP-04.** In production the service shall refuse to start without a database URL, a JWT secret, and a token HMAC secret of at least 16 characters.
- **OTHER-DEP-05.** GitHub Actions shall run the Flutter analyzer and tests, the backend lint, type check and tests (with a Postgres service), and a check for committed secrets on every change.
- **OTHER-DEP-06.** The project shall publish a signed Android APK for pilot installation.

5.2 Legal and Privacy

- **OTHER-PRV-01.** Pilot participants shall learn what the app stores before they join a classroom.
- **OTHER-PRV-02.** Deleting a user shall delete their profile, enrollments, devices, and attendance through cascading deletes.

5.3 Documentation

- **OTHER-DOC-01.** The repository shall hold this SRS in Markdown (`docs/srs/`) and LaTeX (`project-srs/`), the project proposal, UML diagrams, and a deployment runbook.
- **OTHER-DOC-02.** The API shall publish its own reference at `/docs`.

A Glossary

Term	Meaning
BLE	Bluetooth Low Energy
RSSI	Received signal strength indicator, in dBm. Closer devices read less negative values
Epoch	A fixed length time slice used to rotate tokens
Alias	Four random bytes that identify a live session in a token
Challenge	A single use nonce the server issues after validating a token
Claim	A signed statement from a student device that it observed a valid token
RLS	Row level security in Postgres
JWT	JSON Web Token issued by Supabase Auth
SPKI	SubjectPublicKeyInfo, the DER format for public keys

B API Summary

Method and path	Role	Purpose
`POST /v1/profile`	any user	Create the caller's profile
`GET /v1/me`	any	Read the caller's profile
`POST /v1/classrooms`	teacher	Create a classroom
`GET /v1/classrooms`	any	List own classrooms
`POST /v1/classrooms/join`	student	Join with a join code
`PUT /v1/devices/me`	student	Bind a device signing key
`POST /v1/sessions`	teacher	Start a session
`POST /v1/sessions/{id}/end`	teacher	End a session
`GET /v1/sessions/{id}/token`	teacher	Current rotating token
`GET /v1/sessions/{id}/roster`	teacher	Live roster
`POST /v1/sessions/{id}/corrections`	teacher	Manual correction
`GET /v1/sessions/{id}/export.csv`	teacher	CSV export
`POST /v1/attendance/challenge`	student	Trade a token for a nonce
`POST /v1/attendance/claim`	student	Submit a signed claim
`GET /healthz`, `GET /readyz`	none	Liveness and readiness

C Rejection Reasons

The API returns HTTP 409 with `{"status": "rejected", "reason": "<code>"}`.

Code	Meaning	Stage
invalid_token	Malformed token or wrong tag	challenge
token_expired	Token epoch outside the grace window	challenge
session_not_active	Unknown alias or ended session	challenge, claim
not_enrolled	Student is not in the classroom	challenge
no_device_bound	Student has no active device key	challenge, claim
challenge_unknown	Nonce missing or issued to someone else	claim
challenge_expired	Nonce older than 120 seconds	claim
nonce_reused	Nonce consumed by a different claim	claim

Code	Meaning	Stage
token_mismatch	Claim token differs from the challenge token	claim
bad_signature	Signature does not verify against the active key	claim
claim_stale	Observation time more than 60 seconds from server time	claim
too_few_samples	Sample count below the session minimum	claim
rssi_below_threshold	Median RSSI weaker than the session threshold	claim
duplicate	The student already has a record in this session	claim

D Evaluation Plan and Data Schema

Primary metric. Detection success rate: the fraction of labelled position scan attempts (front, middle, back, door, corridor, adjacent room) that accept or reject correctly under the configured policy.

Secondary metrics. Discovery latency, attendance completion latency, false accept and false reject rates at out of room and in room positions, API p50 and p95 latency, and the rejection rate for duplicate claims and expired tokens.

Method. One teacher phone and at least two student phones of different models, in a real classroom. Record raw rows before computing any summary.

Raw CSV schema (`docs/evaluation/*.csv`):

Column	Meaning
run_id	Identifier for one scenario run
timestamp_utc	ISO 8601 time of the attempt
teacher_phone, student_phone	Device models
android_version	Version on the student phone
room_label	Room name
position_label	front, middle, back, door, corridor, adjacent_room
expected	accept or reject
median_rssi_dbm, sample_count	Values sent in the claim
result	accepted or the rejection reason
discovery_ms, completion_ms	Latencies

E Test Traceability

Backend tests live in `backend/tests/`. The suite runs against a real Postgres.

Requirements	Tests
FR-ACC-01 to FR-ACC-05	`test_teacher_and_accounts.py` (allowlist, roll number, idempotent profile, bad credentials, role separation)
FR-CLS-01 to FR-CLS-03, FR-DEV-01, FR-DEV-02	`test_teacher_and_accounts.py` (join code format, listing, unknown code, invalid key), `test_attendance_flow.py` (rebinding revokes the old key)
FR-SES-01 to FR-SES-05, FR-SES-07	`test_tokens.py`, `test_teacher_and_accounts.py` (one active session, ownership, rotation)
FR-ATT-02 to FR-ATT-08	`test_attendance_flow.py`, `test_claims.py`, `test_crypto.py`
FR-ROS-01, FR-COR-01, FR-COR-02	`test_teacher_and_accounts.py` (corrections, roster)
FR-EXP-01 to FR-EXP-03	`test_teacher_and_accounts.py` (CSV contents, formula injection)

Requirements	Tests
FR-AUD-01, FR-AUD-02	`test_attendance_flow.py` (rejections and acceptances audited), `test_teacher_and_accounts.py` (export audited), `test_rls.py`
OTHER-DEP-04	`test_config.py`
NFR-SEC-03	`test_rls.py`
NFR-SEC-05	`test_attendance_flow.py` (tampered RSSI breaks the signature)
Reliability (4.3)	`test_attendance_flow.py` (concurrent claims never double count)

Requirements marked Planned have no automated test yet. The Flutter unit tests live in `code/test/`.